

Backend Developer

UZAIR HUSSAIN SIDDIQUI :: B23110106078

Say this first

I built the API, and my algorithm is **Levenshtein edit distance**.

It counts the smallest number of letter changes needed to turn one text into another. I use it to catch people who copy an answer and rename a few words to hide it.

Time is **$O(m \text{ times } n)$** . Space is **$O(n)$** .

The five facts to memorise

Algorithm	Levenshtein edit distance
Method	Dynamic programming, two rows at a time
Time	$O(m \text{ times } n)$
Space	$O(n)$
My files	main.py and algorithms/levenshtein.py

What I built

The API	32 endpoints. Every screen on the site calls them.
The rule	Routes are thin. Check who is asking, call one module, return JSON. No logic in main.py.
Why that matters	All 14 tests run with no server and no database.
My algorithm	Edit distance, and the copy check built on it.

The algorithm

EDIT DISTANCE :: ONE EXAMPLE, FOUR STEPS

The question it answers

How many single letter changes turn one text into another? You may insert a letter, delete a letter, or replace a letter.

The example everybody asks for

kitten to sitting

1. replace **k** with **s**
2. replace **e** with **i**
3. add **g** at the end

Distance = 3. There is a test that checks exactly this.

How the code does it

1

Make a table. Each cell holds the distance between the start of one text and the start of the other.

2

Fill each cell with the smallest of three options: insert, delete, replace. Replace is free if the letters already match.

3

The answer is the bottom right cell.

4

Only keep two rows, never the whole table. That is why space is $O(n)$.

Turning distance into a score

score = 1 – (distance / length of the longer text)

Same text gives 1. Nothing in common gives 0.

Where I use it

CATCHING THE COPY THAT CHANGED A FEW WORDS

The problem

Rabin Karp finds copied text by matching exact chunks. Rename one variable and the chunk no longer matches.

A real case from our demo data

Somebody copied the winning answer and renamed `items` to `cart_items`, and `total` to `sum`.

Checker	Said	Verdict
Rabin Karp, exact chunks	61 percent	Missed most of it
My edit distance	83 percent	Caught it

So the Copied score is the higher of the two

Copied = max(Rabin Karp, edit distance)

Three things I decided, and why

Rabin Karp runs first	Mine is slow. It only runs on pairs the fast check already flagged.
A floor of 0.6	Any two pieces of code share letters and score about 0.25. Below the floor is noise, so it counts as zero.
Only look backwards	Each entry is compared only with entries sent before it. The first person to submit can never be blamed for being copied.

What that looks like on screen

Ghost	Copied	Final	Why
Quiet Falcon	0.00	0.582	Original, submitted first
Pale Otter	0.13	0.447	Was copied from, barely touched
Calm Lynx	0.00	0.358	Original, but no code
Swift Heron	0.88	0.060	Copied from Pale Otter

Questions

SHORT ANSWERS. SAY THEM AND STOP.

Show kitten to sitting.	Replace k with s. Replace e with i. Add g. Three.
Why dynamic programming?	Recursion redoes the same work over and over. A table does each piece once.
Why only two rows?	Each row only needs the one above it. The rest are never read again.
Why not use your algorithm alone?	It is slow, $O(m \text{ times } n)$ per pair. The fast check narrows it down first.
Why the 0.6 floor?	Unrelated code still scores about 0.25 just from shared letters. That is noise.
Does the original author get punished?	No. Entries are only compared with earlier ones, so whoever submitted first scores zero.
Why is your API thin?	No logic in it, so all 14 tests run without a server or a database.
Two people submit at once?	Each gets its own connection and transaction. The database rejects duplicates.
Why FastAPI?	It writes the API documentation itself from the type hints.

What to open, in this order

1	The /docs page. Every endpoint, generated automatically.
2	Log in as the company, open the cart challenge, press Rank entries.
3	Point at the Copied column: 0.88 for the copier, 0.13 for the original.
4	Open ranker.py and show the eight lines that combine both checks.

If they push on the Quality column

It is the weakest of the four measures and I know it. Comparing an answer to the question mostly tells you if it reuses the same words, and Relevance already does that better. It is the lowest weighted of the three positive measures.